

## System Documentation: Shirsendu's Digital photography album

**Author / Director:** Shirsendu Mondal **Architecture:** Single-Page Application (SPA) **Tech Stack:** HTML5, CSS3, Tailwind CSS (CDN), Vanilla JavaScript, Supabase (Backend-as-a-Service) (dropbox as music storage cloud / stream) and claudinary for image storing cloud- (CDN)

### 1. Executive Summary

This application is a highly interactive, cinematic digital photography portfolio. Rather than acting as a simple static gallery, the site functions as an immersive digital environment. It combines a minimalist "Darkroom" aesthetic with signature amber visual accents, dynamic custom cursors, a continuous background soundtrack, integrated mini-games, and a secured "Special Tab" for private memories.

Under the hood, it uses Supabase to handle real-time community engagement (likes and comments) while heavily utilizing the browser's local storage and session storage to manage user state, authentication, and offline capabilities.

### 2. Visual Architecture & UI Engine (HTML/CSS)

The visual layer is built using a hybrid approach of Tailwind CSS for structural utility classes and raw CSS for complex, high-performance animations.

- **Theming System:** The site relies on native CSS variables (e.g., `--bg-color`, `--card-bg`, `--text-accent`). It includes a toggle function (`toggleGalleryTheme()`) that switches between a "Darkroom" (default) and "Daylight" mode by swapping out these variables dynamically.
- **True CSS Masonry:** The gallery uses native CSS column counts (`column-count`, `break-inside: avoid`) to create a fluid masonry grid that seamlessly adapts to mobile, tablet, and desktop screens without needing a heavy JavaScript library.
- **The Custom Lens:** On desktop screens (min-width: 1024px), the default mouse cursor is hidden and replaced by a custom HTML div (`#custom-lens`). JavaScript tracks the mouse coordinates to move this lens, dynamically changing its color, size, and text (like saying "view") when hovering over interactive elements.
- **Cinematic Reveal & Animations:** The site uses Intersection Observers (`runScrollObserver`) to detect when images enter the viewport, triggering a smooth, staggered fade-in effect (`.reveal-on-scroll`).

### 3. Core Systems & Logic (JavaScript)

The JavaScript acts as the brain of the portfolio, divided into several distinct "Engines."

#### A. The Greeting Engine

When a user first visits, they are met with a "Welcome Gate."

- The script asks for a name, saves it to the browser's localStorage (so they don't have to enter it again on future visits), and triggers a timed, cinematic text sequence using staggered CSS class additions.
- This name is later used to tag any comments they leave on your photos, ensuring a personalized experience.

## B. The Gallery Builder

Instead of writing out hundreds of HTML tags manually, the code relies on a massive data array (myImagesCollection) containing over 200 objects.

- The buildGalleryGrid() function loops through this array, generating the HTML for each image card on the fly.
- It automatically hooks up the double-tap to like, the click to view (Lightbox), and fetches the Supabase comment/like counts for every single photo.

## C. The Cinematic Audio Engine

The site features a built-in background music player that reads from a 44-song array (myPlaylist).

- **Auto-Play & Loop Logic:** An ended event listener watches the audio tag. When a song finishes, playNextTrack() calculates the current index, adds one, and triggers the next song. If it hits the end of the list, it loops back to the first track.
- **Progress Slider & Controls:** The timeupdate event calculates the song's current progress as a percentage and updates the custom slider width and the minute/second timestamps. Users can also manually skip tracks or pause using the custom floating controller.

## D. The Engagement Engine (Likes & Comments)

This connects your front-end code to your backend Supabase database for real-time interaction.

- **Bulletproof Likes:** To prevent a single user from spamming a photo with thousands of likes, triggerGlobalLike() uses sessionStorage to cap likes at **10 per visit**. Once verified, it calls a Supabase RPC (Remote Procedure Call) named increment\_likes to safely update the global database.
- **Comments Drawer:** Clicking the comment button slides out a side-panel. It fetches existing comments from the comments table based on the photoId and displays them. If the user submits a new comment, it inserts their text and localStorage username into the database, instantly refreshing the view with color-coded names.

## 4. Special Effects & Easter Eggs

To make the portfolio feel alive, several background effects run autonomously:

- **Hardware Confetti:** Clicking the "Like" button triggers a canvas-based particle simulation (`stepConfettiSimulationFrame()`). It calculates gravity, speed, and opacity to shoot colorful pixels across the screen.
- **Realistic Pigeon Engine:** A highly complex mathematical function `launchRealisticPigeons()`. It spawns an image of a pigeon, adds flapping mechanics using `Math.sin(time)`, adjusts scale for depth, plays a synchronized wing-flapping audio clip, and randomly drops trailing CSS "feathers" that drift down the screen.
- **Engagement Timer:** A hidden script runs a `setTimeout` for 10 minutes (600,000 milliseconds). If a user stays on the site that long, a "Dedicated Explorer" modal pops up to thank them. Acknowledging it triggers massive confetti and pigeon animations.
- **End-of-Site Modal:** A scroll event listener watches to see if the user hits the absolute bottom of the page. Once triggered, it locks the screen and displays a friendly "Oops! You've reached the end" message, encouraging them to look forward to new uploads.

## 5. The Locked Vault (Special Tab)

A hidden, highly stylized section of the site secured by a JavaScript logic gate.

- **Authentication:** The user must enter the exact string ..... into the `#vaultPass` input to bypass the security screen.
- **The Togetherness Meter:** Once unlocked, a JavaScript `setInterval` calculates the exact difference in Days, Hours, Minutes, and Seconds between the current precise time and a milestone date (December 2, 2024). It updates a beating digital heart on the screen every 1000 milliseconds.
- **Private Masonry:** A secondary gallery (`.photo-fluid-matrix`) loads exclusively inside this secure environment, utilizing a staggered CSS cascade animation (`triggerCardStaggerCascade()`) for a grand visual reveal.

## 6. The Arcade Interface

A built-in overlay containing three playable logic games to keep visitors engaged.

1. **Tic-Tac-Toe:** A Player vs. Computer match. The AI uses an array-matching algorithm (`calculateAICounterMove()`) to check winning combinations, block player wins, or logically pick open squares.
2. **Sudoku:** Generates a 9x9 grid based on a hardcoded array. An `oninput` function watches the user's typing; if the number matches the hidden solution array, it highlights the box in emerald green dynamically.
3. **Cinematic Snake Run:** A fully functional canvas game. The game loop runs every 160ms. It calculates grid coordinates, listens for Arrow Key inputs to change direction, detects self-collision

(tail biting), handles dynamic score tallying, and even replaces the "Snake Head" with a custom avatar image.